Intoxalock Device-Removal Scheduling — Solution Architecture & System Behavior

> Today, scheduling a device removal is a manual, phone-based coordination task between an Intoxalock representative and an independent service center that has no availability data and no scheduling API. Requests are slow to confirm, silently dropped when a center doesn't answer, and routinely mistaken by customers for real appointments. The proposed solution replaces that manual coordination with a **single, long-running, event-driven workflow** on AWS: each removal request becomes one automated process that waits until the center is open, **calls the center with an AI voice agent** to confirm the appointment and capture the quote, **reaches the customer by SMS + email** only when a new time must be agreed, **issues a real work order**, confirms the customer, and **escalates to a human — with full history — only on true exceptions**. The result is an ambiguous, drop-prone request turned into a reliably-confirmed appointment backed by a valid work order.

This document explains the proposal from three complementary angles, each with its own diagram:

1. **What the system does** — the end-to-end business flow a request travels through (Figure 1).
2. **Who owns what** — the split of responsibilities between Intoxalock and Source Meridian, and the exact API contracts on the seam between them (Figure 2).
3. **How it runs on AWS** — the technical architecture and runtime behavior (Figure 3).

---

1. What the system does — the end-to-end flow

Figure 1 shows the full journey of a single removal request across every participant: the **Customer**, the **intake channels** (app / IVR), the **Automation System**, the independent **Service Center**, and — only on exceptions — an **Intoxalock Representative**. The right-hand column calls out, phase by phase, what each step fixes versus today.

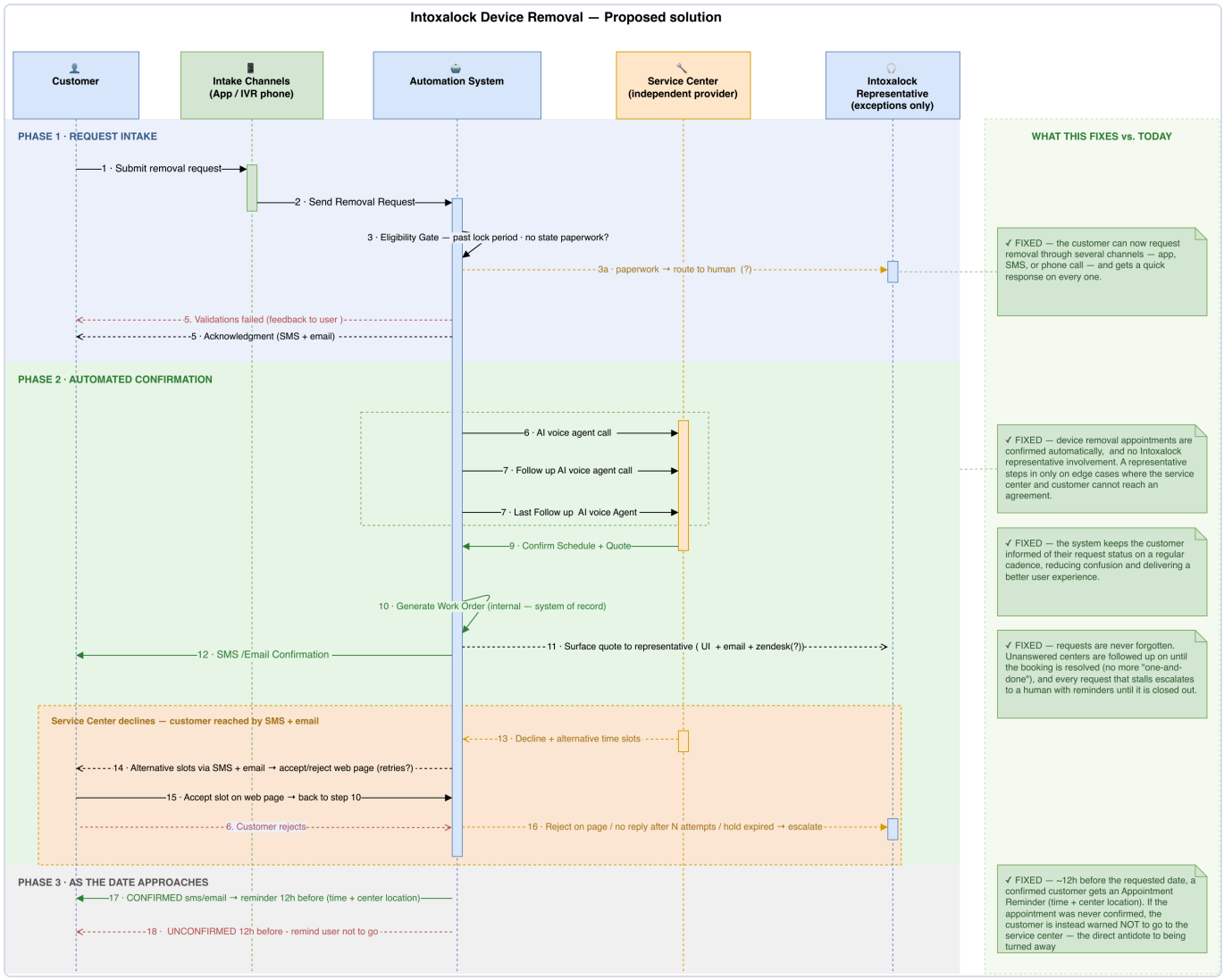


Figure 1 — Proposed solution, end-to-end business flow (three phases).

Phase 1 · Request Intake. The customer submits a removal request through their preferred channel (app or IVR phone). The Intoxalock backend checks the request is eligible for automation — it is **past the lock period** and needs **no state paperwork** — and immediately sends the customer an acknowledgment (SMS + email). Requests that need paperwork are routed to a human; requests that fail validation get clear feedback. Only clean, eligible requests move forward.

Phase 2 · Automated Confirmation. This is the heart of the system and runs with **no representative involvement**:

- An **AI voice agent calls the service center** to confirm the appointment and capture the vehicle-based quote. If the center doesn't answer, the agent **follows up and retries** rather than giving up.
- On a clean **confirmation**, the system **generates the work order** (the internal system of record) and sends the customer an **SMS/email confirmation with the quotation**. The quote is also surfaced to the representative for visibility.
- If the center **declines and offers alternative times**, the customer is reached by **SMS + email** with a link to an **accept/reject web page**. Accepting a new slot flows straight into the work-order step; rejecting, not

replying after N attempts, or letting the hold expire **escalates** the request to a human.

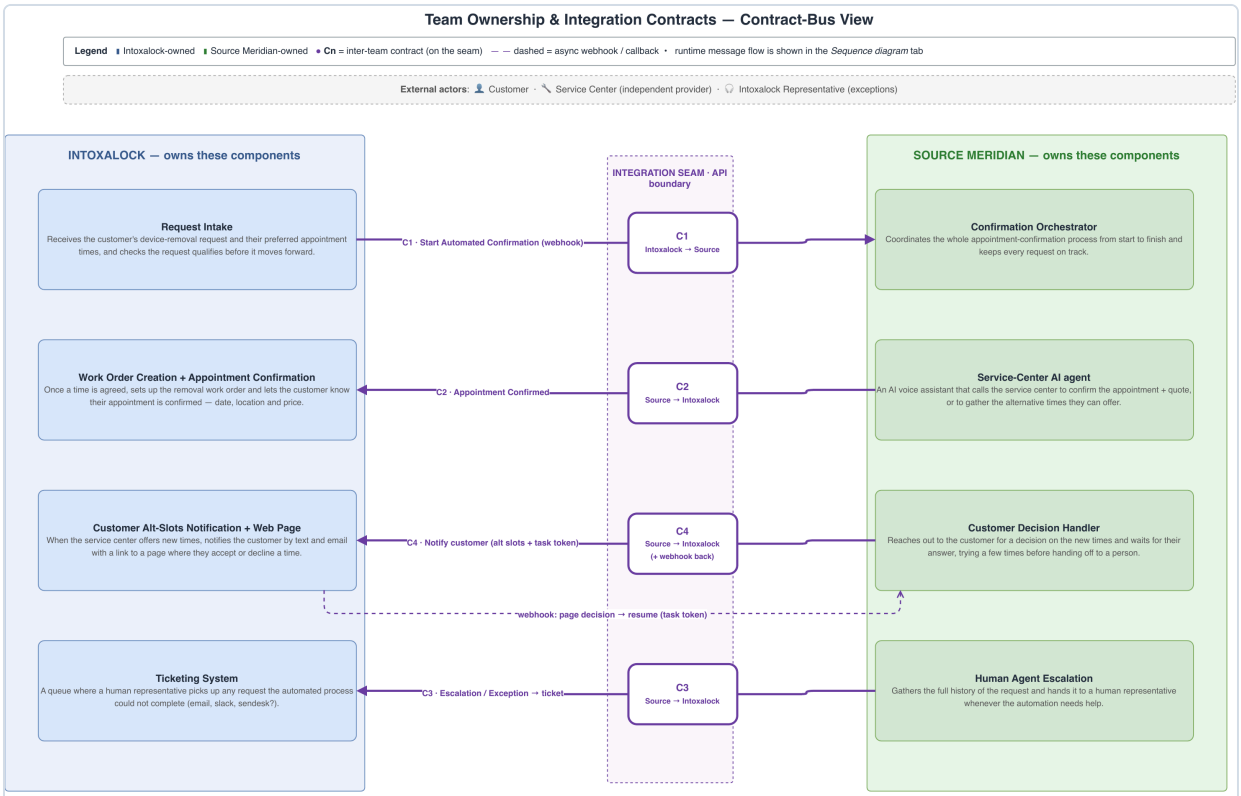
Phase 3 · As the date approaches. A **confirmed** customer receives an appointment reminder ~12 hours ahead with the time and center location. If a request was **never confirmed**, the customer is instead warned **not to go** — the direct antidote to the "showed up to a closed or unaware center" failure that happens today.

What this fixes. Requests are never forgotten (unanswered centers are followed up until resolved), confirmation is unambiguous (it is tied to a real work order), the customer is kept informed on a steady cadence, and a human only ever steps in on genuine edge cases.

---

2. Who owns what — responsibilities and integration contracts

The system is delivered as a partnership. Figure 2 (the *Contract-Bus View*) draws a clean line down the middle: **Intoxalock-owned** components on the left, **Source Meridian-owned** components on the right, and the **integration seam** — the small set of API contracts (C1–C4) that cross the boundary — in the center, with the full payload and trigger for each contract in the registry at the bottom of the diagram. Everything else is an internal implementation detail of whichever side owns it.



Integration Contract Registry

#	Contract	Trigger	Payload & Response (key fields)
C1	Start Automated Confirmation (webhook)	Intake passes the Eligibility Gate.	**Request** `{ "customer": { "id": "string", "phone": "E.164" , "email": "string" , "name": "string" , }, "assigned_service_center": { "id": "string", "phone": "E.164", "name":"string", timezone : "string", bussines_hours : "string" }, "customer_prefered_time_slots": ["string", "string"], "vehicle_information": { "make": "string", "model": "string", "year": 2020 }, "eligibility": true }` **Response** — 202 `{ "request_id": "uuid" }`

C2	Appointment Confirmed (→ Work Order + notify)	Appoiment confirmation with quote .	**Request**
			`{ "request_id": "uuid", "confirmed_date_time": "string", "customer_id" : "string" "service_center_id": "string", "quote": { "amount": float }, "scheduled_time_slot" :"string" }`
			Response — 200
			`{ "work_order_id": "string" }`

C3	Exception / Escalation	No agreement, TTL expiry.	**Request** `{ "requestId": "uuid", "customer": { "id": "string", "phone": "E.164", "email": "string", "name": "string" }, "assigned_service_center": { "id": "string", "phone": "E.164", "name": "string" }, "reason": "NO_AGREEMENT | MAX_SERVICE_CENTER_RETRIES | MAX_CUSTOMER_RETRIES | CUSTOMER_REJECTED | TTL_EXPIRED", "last_state": "string", "full_history": [{ "time_stamp": "string", "channel": SMS | EMAIL | AI_VOICE_AGENT | WEB_PAGE, "event": "string" , result : SUCCESS | FAILURE}] }` **Response** — 200 `{ "ticket_id": "string" }`

C4	Notify Customer — Alternative Slots + Web Page (waitForTaskToken)	Service center declined and offered alternative slots (Intoxalock sends SMS + email & hosts the accept/reject page).	**Request** — Source → Intoxalock notify API
			`{ "request_id": "uuid", "customer": { "id": "string"}, "alternative_slots": [ "string", "string" ], "quote": { "amount": float }, "task_token": "string", "expires_at": "ISO-8601" }`
			Response — 202
			Webhook — Intoxalock web page → Source (resumes token)
			`{ "request_id": "uuid", "task_token": "string", "decision": "ACCEPTED | REJECTED", "selected_slot": "string | null" }`

Figure 2 — Team ownership and the four integration contracts on the seam.

Intoxalock owns the systems of record and the customer touchpoints it already operates: **Request Intake**, **Work Order Creation + Appointment Confirmation**, the **Customer Alternative-Slots notification + web page**, and the **Ticketing System** that a human uses to pick up escalations.

Source Meridian owns the automation brain: the **Confirmation Orchestrator** (the long-running workflow that keeps every request on track), the **Service-Center AI agent** (the voice assistant that calls centers), the **Customer Decision Handler** (which reaches the customer for a decision on new times), and the **Human-Agent Escalation** step (which assembles full history and hands off).

Only **four contracts** cross the seam — keeping the interface this small is deliberate, so each side can evolve its internals freely as long as these four hold. **C4** is the one two-way contract: Source hands Intoxalock a `task_token`, Intoxalock hosts the customer's accept/reject page, and the customer's decision is **webhooked back** to resume the workflow exactly where it paused.

---

3. How it runs on AWS — architecture and system behavior

The problem is defined by **waiting, external events, and unreliable participants** — exactly what event-driven orchestration is built for. Nothing is polled or manually chased: the system places a call or sends a notification, then **pauses** until the real-world outcome — a call ending, a customer's decision on a web page — arrives as an **event**, and resumes exactly where it left off. This is why the workflow is built on **AWS Step Functions**: it suspends long waits at **zero compute cost**, resumes natively on external callbacks (`waitForTaskToken`), and provides built-in retries plus a full, visual execution history — so every request's status is always visible and nothing is silently dropped.

The Confirmation Orchestrator is implemented as a **single AWS Step Functions (Standard Workflow)** state machine: **one execution per removal request**. Figure 3 is the full technical architecture — intake edge, the state machine and its stages, the reusable Outbound Call Service, and the data stores.

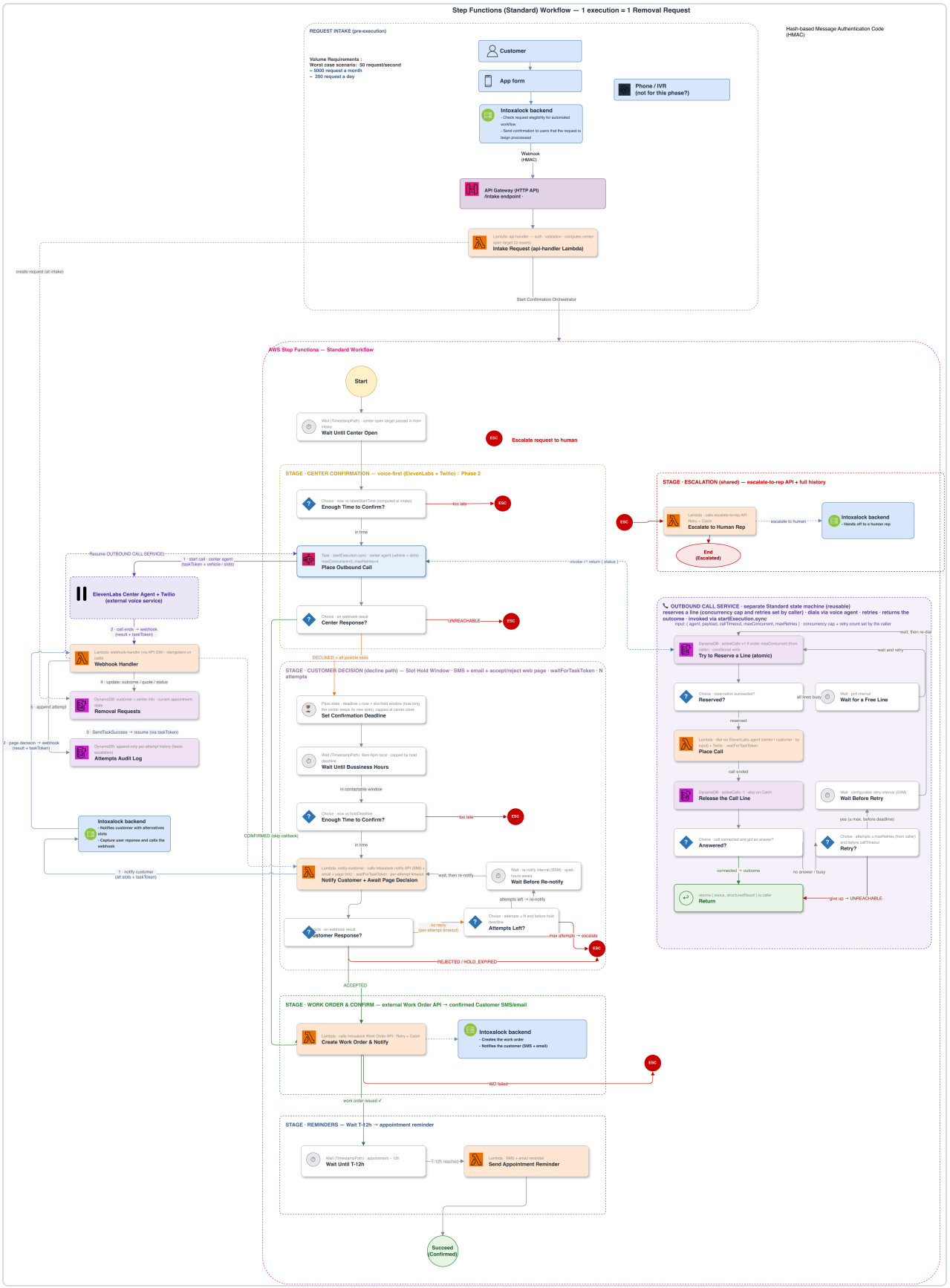


Figure 3 — AWS Step Functions architecture (one execution = one removal request).

3.1 The workflow, stage by stage

Pre-execution · Request Intake. Before the workflow starts, the Intoxalock backend posts the request to **API Gateway → `api-handler` Lambda**, which validates it, computes the time-zone-aware **"center-open" target**, creates the **Removal Requests** record, and starts the state machine. The workflow therefore only ever handles eligible, well-formed requests and never has to reason about intake validation at runtime.

Stage 1 · Wait Until Center Open. A **Wait** state holds the execution — at **zero compute cost** — until the center's working hours. Nothing is polling; the workflow simply sleeps until it is actually useful to call.

Stage 2 · Center Confirmation (voice-first). A lead-time guard (*Enough Time to Confirm?*) escalates if it is already too late to make the window. Otherwise the workflow invokes the **Outbound Call Service** with the **center agent** and the vehicle/slot payload; the agent confirms the appointment and captures the quote. On the result: `CONFIRMED` → straight to the work order; `DECLINED + alternative slots` → the Customer Decision stage; `UNREACHABLE` after retries → escalate.

Stage 3 · Customer Decision (decline path). Used only when the center offered different times. The workflow sets a **Slot-Hold deadline** (how long the center will hold the new slots, capped at center close), re-checks the runway, then reaches the customer via **SMS + email** (the Notify contract, C4) and **pauses** for the accept/reject page decision. `ACCEPTED` → work order; `REJECTED / HOLD_EXPIRED / no reply after N attempts` → escalate.

Stage 4 · Work Order & Confirm. A single Lambda calls the **Intoxalock Work Order API**, which creates the de-installation work order and notifies the customer (SMS + email). This guarantees the customer only ever receives a confirmation that is **backed by a real work order**.

Stage 5 · Reminders. *Wait Until T-12h → Send Appointment Reminder →* **Succeed (Confirmed)** — the terminal happy path.

Escalation (shared). Any unrecoverable branch routes to one shared path: a single Lambda calls the **escalate-to-rep API** with the **full attempt history** and ends in **End (Escalated)**. Escalation is now the exception, not the default — and the human starts with a complete record instead of from scratch.

3.2 Architecture components

Component	Role
Step Functions (Standard)	The durable orchestrator. One execution per request; waits hours/days at no compute cost; native retries, catch, and full execution history.
API Gateway + `api-handler` Lambda	Intake edge: validate, compute the center-open target, create the request record, start the execution.

Component	Role
Outbound Call Service (nested state machine)	Encapsulates the whole "place a capacity-gated, retried voice call" primitive used to reach the service center. Reused by every execution that needs to place a call, so the capacity logic is defined once.
ElevenLabs + Twilio (voice agent)	External AI voice service — the **center agent** that calls the service center. The call uses `waitForTaskToken`; its outcome resumes the workflow.
Webhook Handler Lambda	Receives async outcome webhooks — the call-end result from the voice agent and the page-decision result from the customer — idempotent on their IDs, resumes the paused step via `SendTaskSuccess`, updates state, and appends to the audit log.
DynamoDB — Removal Requests	Per-request customer/center info and current appointment state.
DynamoDB — Attempts Audit Log	Append-only per-attempt history; feeds escalation.
DynamoDB — Call Capacity counter	A distributed semaphore enforcing a **configurable concurrency cap** on simultaneous outbound calls, shared across all executions.
Intoxalock Work Order API / backend	External system of record: creates work orders, notifies customers, and handles human escalation.
SSM Parameter Store	Tunable knobs: concurrency cap, retry count/interval, slot-hold window.

3.3 The reusable Outbound Call Service

Every execution that needs to reach a service center goes through the exact same mechanics: reserve a line under the shared, **configurable** concurrency cap, dial, wait for the call to end, retry on no-answer, and release the line (even on failure). Extracting this into **one child state machine** means that logic — and the correctness of the shared concurrency semaphore — is defined **once** and stays consistent no matter how many requests are calling out at the same time. Callers pass `{ agent, payload, callTimeout, maxConcurrent, maxRetries }` and receive `{ status, structuredResult }`.